ADVANCED ANALYSIS OF ALGORITHMS

CS5031 — Week 2: Lectures 3–4

Comprehensive Student Notes

Runtime Analysis, Asymptotic Notation, Big-O Notation, and Algorithm Design Techniques

Chapter 1: Introduction to Algorithm Runtime Analysis

1.1 What Is an Algorithm?

An algorithm is a finite sequence of well-defined instructions that solves a particular problem or performs a specific task.

An algorithm accepts some input, processes it according to a set of instructions, and produces the required output.

For example, consider the problem of finding the largest number in a list.

Input: [12, 7, 25, 9, 18]

Output: 25

A simple algorithm to solve this problem is:

1.

Assume the first number is the largest.

Compare it with the next number.

If the next number is larger, update the largest number.

Repeat the comparison until all numbers have been examined.

Return the largest number.

This is an algorithm because its instructions are clear, finite, and produce the correct output.

1.2 What Is Algorithm Analysis?

Algorithm analysis is the process of studying the resources required by an algorithm to solve a problem.

The two most important resources are:

Time complexity: Describes how the amount of computation performed by an algorithm grows as the input size increases.

Space complexity: Describes how the amount of memory required by an algorithm grows as the input size increases.

Consider two algorithms that solve the same problem.

- Algorithm A performs approximately n operations.
- Algorithm B performs approximately n^2 operations.

Here, n represents the input size.

For an input of $n = 100$:

$$T_A(100) = 100$$

$$T_B(100) = 100^2 = 10,000$$

Algorithm B performs many more operations under this simplified model.

As the input size grows, the difference becomes even greater.

This is why algorithm analysis is important: it helps us predict how well an algorithm will perform for large inputs.

1.3 What Is Runtime?

The **runtime of an algorithm** refers to the amount of time required to execute it.

Runtime can be discussed in two different ways:

Actual execution time: The amount of physical time, usually measured in seconds or milliseconds, that a computer takes to execute a program.

Theoretical runtime: A mathematical description of the amount of computation required by an algorithm as a function of input size.

For example, a program might take 0.5 seconds on one computer and 2 seconds on another computer.

However, its theoretical time complexity might be $O(n)$ on both machines.

The distinction is important because actual running time depends on the computer, whereas theoretical complexity helps us study how computational work grows.

1.4 What Is Input Size?

The **input size**, usually represented by n, describes the amount of data or the numerical parameter that determines the amount of work performed by an algorithm.

Examples include:

Problem	**Meaning of n**
Searching a list	Number of elements
Sorting an array	Number of elements
Finding a Fibonacci number	Index of the required Fibonacci number
Processing a string	Number of characters
Processing a matrix	Number of rows or columns, when specified
Graph processing	Number of vertices and/or edges

The meaning of n must be identified before analyzing an algorithm.

An important distinction is that the numerical value of an input is not necessarily the same as the number of bits required to represent that input. For example, the binary representation of an integer n requires approximately $\log_2 n$ bits.

1.5 Fibonacci Numbers

The Fibonacci sequence is a sequence of numbers in which each number, starting from the third, is obtained by adding the previous two numbers.

Mathematically:

$$F(0) = 0$$

$$F(1) = 1$$

For $n \geq 2$:

$$\boxed{F(n) = F(n-1) + F(n-2)}$$

The first several Fibonacci numbers are:

Index n	Fibonacci number $F(n)$
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
9	34
10	55

For example:

$$F(5) = F(4) + F(3)$$

$$F(5) = 3 + 2 = 5$$

Similarly:

$$F(6) = F(5) + F(4)$$

$$F(6) = 5 + 3 = 8$$

Fibonacci numbers provide a useful example for understanding arrays, loops, addition operations, and runtime analysis.

1.6 Computing Fibonacci Numbers Using an Array

One method for calculating Fibonacci numbers is to store previously calculated values in an array.

This avoids repeatedly calculating the same Fibonacci numbers.

Algorithm: FibList

```
Plain text
Function FibList(n)
    create an array F[0 ... n]
    F[0] ← 0
    F[1] ← 1
    for i from 2 to n:
        F[i] ← F[i-1] + F[i-2]
    return F[n]
```

This version assumes that $n \geq 1$. For $n = 0$, the algorithm can return 0 immediately.

Explanation of important terms

Function: A named block of instructions that performs a particular task.

Parameter: A value supplied to a function. In `FibList(n)`, the parameter is `n`.

Array: A data structure used to store multiple values, where each value can be accessed through an index.

Index: The position of an element in an array. For example, `F[0]` refers to the first element.

Assignment operator (`←`): Indicates that a value is stored in a variable or array element.

For example:

```
Plain text
F[0] ← 0
```

means that the value 0 is stored in the first position of array `F`.

For loop: A control structure used to repeat instructions for a specified range of values.

Iteration: One execution of the body of a loop.

Return statement: An instruction that sends a result back to the function's caller.

1.7 Dry Run of the FibList Algorithm

A **dry run** is a manual, step-by-step execution of an algorithm using a specific input.

It helps us understand how variables change, how loops execute, and how an algorithm produces its result.

Consider:

$$n = 6$$

We want to calculate:

$$F(6)$$

Step 1: Create the array

```
Plain text

F[0 ... 6]
```

Since indexing starts at 0, the array contains seven positions.

```
Plain text

Index:    0   1   2   3   4   5   6
Value:    _   _   _   _   _   _   _
```

Step 2: Initialize the first two elements

```
Plain text

F[0] ← 0
F[1] ← 1
```

The array becomes:

```
Plain text

Index:    0   1   2   3   4   5   6
Value:    0   1   _   _   _   _   _
```

Step 3: Start the loop

The loop starts at `i = 2` and continues until `i = 6`.

Iteration	i	Calculation	New value
1	2	$F[2] = F[1] + F[0] = 1 + 0$	1
2	3	$F[3] = F[2] + F[1] = 1 + 1$	2
3	4	$F[4] = F[3] + F[2] = 2 + 1$	3
4	5	$F[5] = F[4] + F[3] = 3 + 2$	5
5	6	$F[6] = F[5] + F[4] = 5 + 3$	8

Step 4: Examine the final array

```
Plain text
Index:   0   1   2   3   4   5   6
Value:   0   1   1   2   3   5   8
```

Step 5: Return the result

```
Plain text
return F[6]
```

Therefore:

$$\boxed{F(6) = 8}$$

1.8 Counting the Number of Loop Iterations

The loop is:

```
Plain text
for i from 2 to n:
```

The variable `i` starts at 2 and ends at n, including both endpoints.

Therefore, the number of iterations is:

$$n - 2 + 1$$

$$\boxed{n-1}$$

For $n = 6$:

$$6 - 1 = 5$$

The loop executes five times.

For $n = 10$:

$$10 - 1 = 9$$

The loop executes nine times.

For large values of n, the number of iterations grows approximately in direct proportion to n.

This is called **linear growth**.

1.9 Why Counting Lines of Code Is Not Enough

A common mistake is to assume that every instruction takes exactly the same amount of time.

Consider:

```
Plain text
x ← 5
```

and:

```
Plain text
F[i] ← F[i-1] + F[i-2]
```

Both are written as one line, but they do not necessarily require the same amount of computation.

The first instruction assigns a constant value to a variable.

The second instruction involves:

- Finding two array elements.
- Reading their values.
- Adding the values together.
- Storing the result in another array position.

Furthermore, Fibonacci numbers can become extremely large. Adding two very large integers may require more computational effort than adding two small integers.

Therefore, **the number of executed source-code lines is not necessarily equal to the actual runtime**.

For introductory analysis, we often count basic operations and assume that each operation takes constant time. This produces a simplified model that is useful for comparing algorithms.

1.10 Operations Performed by the Fibonacci Algorithm

Instruction	Operations involved
Create array	Memory allocation and possible initialization
`F[0] ← 0`	Assignment
`F[1] ← 1`	Assignment
`for i from 2 to n`	Initialization, comparison, increment, branching
`F[i] ← F[i-1] + F[i-2]`	Two array accesses, addition, assignment
`return F[n]`	Array access and return

Let us understand these operations individually.

Memory allocation means reserving storage space for data.

Array lookup means accessing an element through its index.

Increment means increasing a variable, usually by one. For example:

$$i \leftarrow i + 1$$

Comparison means checking a condition such as:

$$i \leq n$$

Branching means deciding which instruction to execute next, depending on a condition.

Integer addition means calculating the sum of two integers. With fixed-size machine integers, addition is usually treated as a constant-time operation in the standard RAM model.

1.11 Detailed Loop-Control Dry Run

Consider again:

```
Plain text
for i from 2 to 6:
    F[i] ← F[i-1] + F[i-2]
```

The loop does more than calculate five Fibonacci values.

It must also test whether each iteration should continue.

Step	Value of i	Condition $i \leq 6$	Action
1	2	True	Calculate `F[2]`
2	3	True	Calculate `F[3]`
3	4	True	Calculate `F[4]`
4	5	True	Calculate `F[5]`
5	6	True	Calculate `F[6]`
6	7	False	Exit loop

Notice that the loop body executes five times, but the continuation condition is evaluated six times.

The final comparison is necessary to determine that the loop should stop.

This distinction becomes useful when calculating more detailed operation counts.

1.12 Why Exact Runtime Calculation Is Difficult

Even after examining each instruction, predicting the exact execution time of a program is difficult.

Several factors affect the actual runtime.

A. Speed of the computer

Different computers have different processing capabilities.

A processor may execute some operations faster than another processor.

For example, suppose two computers process the same algorithm.

- Computer A completes an operation in 2 nanoseconds.

- Computer B completes the operation in 5 nanoseconds.

If the algorithm performs one million such operations, the time spent on those operations would be:

Computer A:

$$1,000,000 \times 2 = 2,000,000 \text{ ns}$$

Computer B:

$$1,000,000 \times 5 = 5,000,000 \text{ ns}$$

The algorithm is the same, but the hardware causes different execution times.

B. System architecture

Computer architecture describes how the major components of a computer are organized and communicate.

Important components include:

Control Unit (CU): Coordinates the execution of instructions.

Arithmetic Logic Unit (ALU): Performs arithmetic operations, such as addition, and logical operations, such as comparisons.

Accumulator: A processor register used to store intermediate values during certain computations.

Memory: Stores instructions and data required by programs.

Input devices: Allow information to enter the computer.

Output devices: Present or transmit the results of processing.

Two computers may have different processor architectures, instruction sets, and methods for transferring data between memory and the processor.

These differences can affect runtime.

C. Compiler being used

A **compiler** is a software tool that translates source code into another representation, commonly machine code, so that it can be executed.

Source code is the human-readable program written in a programming language.

Machine code consists of instructions that can be executed by a processor.

A compiler may perform optimizations such as:

- Removing unnecessary instructions.
- Simplifying mathematical expressions.
- Rearranging instructions.
- Improving the execution of loops.

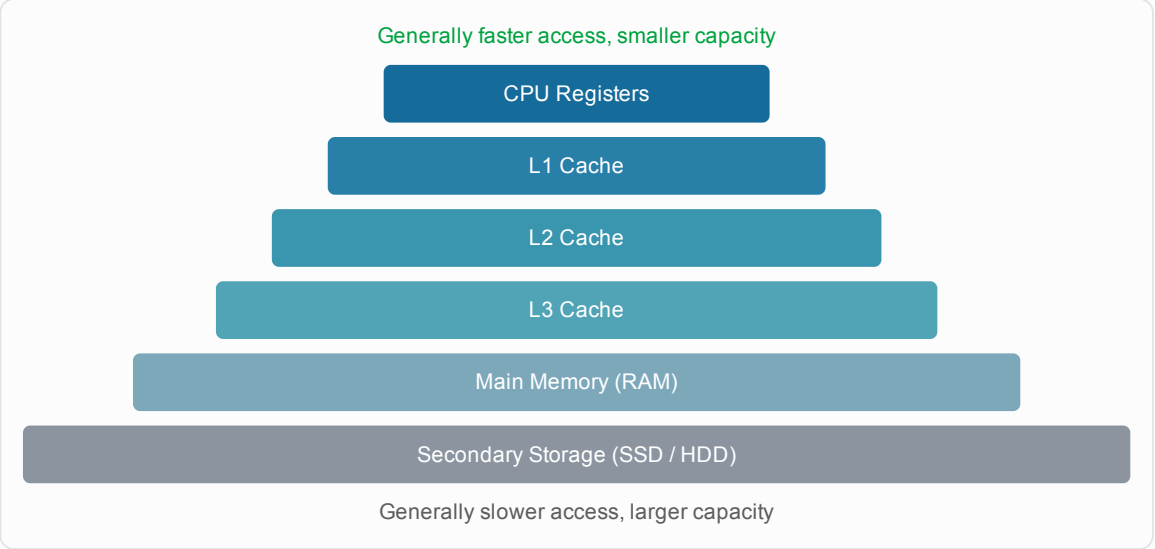
Different compilers, compiler versions, and optimization settings may generate different machine instructions for the same source program.

Consequently, the resulting execution times may differ.

D. Memory hierarchy

Computers use several levels of memory because there is a trade-off between speed, capacity, and cost.

A simplified memory hierarchy is:



Registers: Very small, fast storage locations inside the CPU.

Cache memory: High-speed memory that stores frequently or recently used data and instructions.

L1 cache: Usually the smallest and fastest major cache level.

L2 cache: Usually larger but slower than L1.

L3 cache: Often larger and shared by multiple CPU cores, depending on the processor design.

RAM (Random Access Memory): The main working memory used by running programs.

SSD (Solid-State Drive): A storage device that typically uses flash memory.

HDD (Hard Disk Drive): A storage device that uses magnetic disks.

If required data is present in a fast cache, the processor may access it quickly.

If the data must be retrieved from slower memory, the operation may take longer.

The memory hierarchy is therefore another reason that exact runtime cannot be determined merely by counting program instructions.

E. Memory management

Memory management refers to the way a system allocates, uses, and releases memory.

Creating an array may involve reserving memory, initializing elements, checking available capacity, or other operations.

The time required depends on the programming environment and allocation method.

For example, allocating an array with n initialized elements generally requires work proportional to n. Merely reserving a block of storage may have a different cost.

1.13 The Need for a Simplified Runtime Model

Calculating exact runtime requires detailed information about the program and the computer.

In many situations, these details are unavailable or difficult to model.

Algorithm analysis therefore uses a simpler approach.

Instead of asking:

Exactly how many seconds will this algorithm take on a particular computer?

we ask:

How does the amount of computation grow when the input size increases?

This question leads to **asymptotic analysis**, which is the subject of the next chapter.

Chapter 1 Summary

- An algorithm is a sequence of instructions for solving a problem.
- Runtime describes the computation time required by an algorithm.
- Time complexity studies how computational work grows with input size.
- A dry run helps trace an algorithm step by step.
- A loop's body-execution count is not necessarily the same as its condition-check count.
- Exact runtime depends on hardware, compilers, memory, and the operations performed.
- A mathematical growth-rate model helps us analyze algorithms without knowing every machine-level detail.

Chapter 2: Asymptotic Analysis and Growth Rates

2.1 Introduction to Asymptotic Analysis

Asymptotic analysis is a mathematical technique used to describe how the computational requirements of an algorithm grow as its input size becomes very large.

The word *asymptotic* refers to the behavior of a function as its input approaches a limiting value. In algorithm analysis, we are usually interested in what happens as:

$$n \to \infty$$

Here:

- n represents the input size.
- ∞ represents infinity.
- $n \to \infty$ means that n becomes arbitrarily large.

Asymptotic analysis focuses on the growth rate of an algorithm rather than its exact execution time.

For example, consider two runtime functions:

$$T_1(n) = 5n + 10$$

$$T_2(n) = 3n^2 + 2$$

For small inputs, both functions may produce manageable amounts of work.

However, as n increases, the second function grows much faster because it contains the term n^2.

Asymptotic analysis helps us recognize this difference.

2.2 What Is a Growth Rate?

The **growth rate** describes how quickly a mathematical function increases when its input increases.

Consider three functions:

$$f(n) = n$$

$$g(n) = n^2$$

$$h(n) = 2^n$$

These represent linear, quadratic, and exponential growth, respectively.

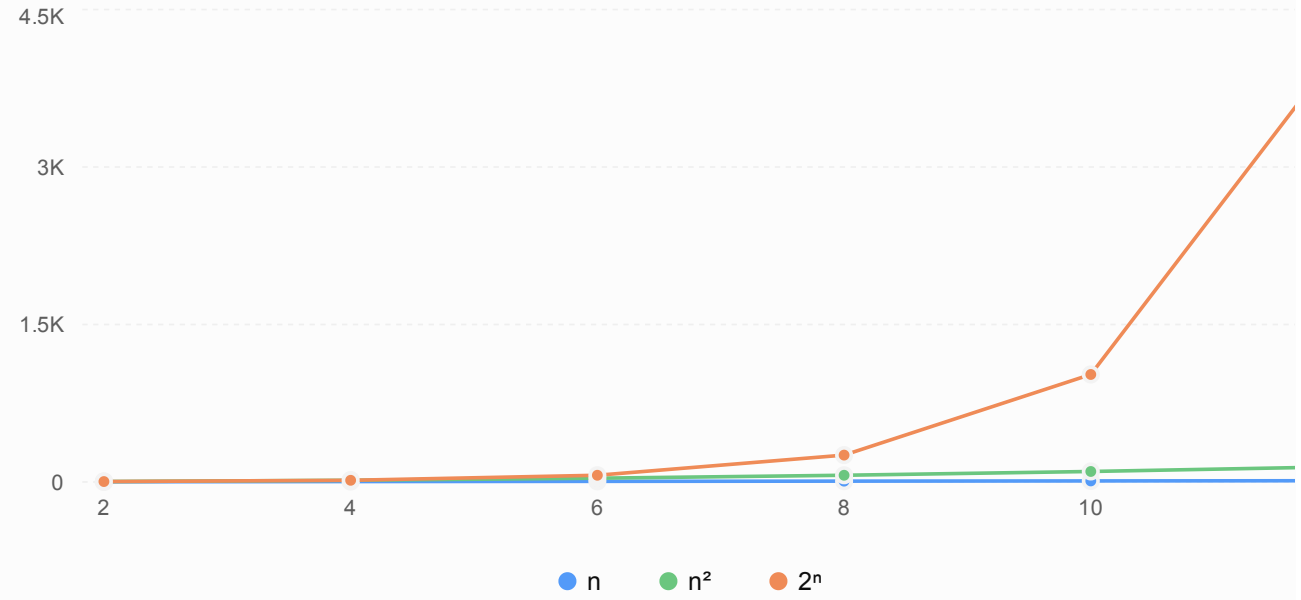
Input size n	n	n^2	2^n
2	2	4	4
4	4	16	16
6	6	36	64
8	8	64	256
10	10	100	1,024
15	15	225	32,768
20	20	400	1,048,576

Notice that 2^n becomes much larger than the other two functions.

The important idea is not the value of a function at one particular input. Instead, we want to understand how quickly the function grows.

Comparison of Linear, Quadratic, and Exponential Growth

A logarithmic-looking increase in separation is shown using exact raw values on a common vertical scale.



In the graph, exponential growth quickly dominates the other growth rates. As the graph's vertical scale expands, the smaller functions may appear nearly flat.

This does not mean that linear and quadratic functions stop increasing. It means that their values are much smaller than the exponential function on the same scale.

2.3 Why Do We Ignore Constant Multipliers?

Suppose two algorithms have the runtime functions:

$$T_A(n) = 5n$$

$$T_B(n) = 20n$$

Algorithm B performs four times as much work as Algorithm A under this model.

However, both functions grow linearly.

For example:

n	$5n$	$20n$
10	50	200
100	500	2,000
1,000	5,000	20,000
10,000	50,000	200,000

The ratio is always:

$$\frac{20n}{5n} = 4$$

The difference is a constant factor.

A **constant factor** is a fixed multiplier that does not change with the input size.

Asymptotic analysis ignores such multipliers because they do not change the category of growth.

Thus:

$$5n = O(n)$$

and:

$$20n = O(n)$$

Both functions have linear growth.

However, this does not mean the algorithms have identical execution times. The factor of four can still matter in practice.

2.4 Why Ignoring Constants Has Limitations

Consider:

$$T_A(n) = n$$

$$T_B(n) = 1000n$$

Both are linear functions.

However, for $n = 100$:

$$T_A(100) = 100$$

$$T_B(100) = 100,000$$

Algorithm B performs one thousand times as much work.

Asymptotic notation deliberately ignores this difference.

Therefore, asymptotic analysis is particularly useful for studying scalability, but it does not fully describe actual performance.

Scalability means how effectively an algorithm handles increasing input sizes.

2.5 Common Classes of Growth

Several mathematical functions occur frequently in algorithm analysis.

A. Constant growth — $O(1)$

An algorithm has constant time complexity when the number of operations is bounded independently of the input size.

Example:

Plain text

```
Function FirstElement(A):
    return A[0]
```

Whether the array contains 10 elements or one million elements, the algorithm accesses one element.

Thus:

$$T(n) = O(1)$$

This assumes ordinary constant-time array indexing.

B. Logarithmic growth — $O(\log n)$

An algorithm has logarithmic complexity when its work grows proportionally to the logarithm of the input size.

A common example is repeatedly dividing a problem size by two.

```
Plain text
Function Halve(n):

    while n > 1:
        n ← floor(n / 2)
```

Here, `floor` means rounding down to the nearest integer.

Dry run for $n = 32$

```
Plain text
32 → 16 → 8 → 4 → 2 → 1
```

The loop executes five times.

Since:

$$2^5 = 32$$

we have:

$$\log_2 32 = 5$$

For powers of two, the number of halvings required to reach 1 is exactly $\log_2 n$.

For other positive integers, the count is approximately logarithmic.

Therefore:

$$T(n) = O(\log n)$$

An important observation is that doubling the input adds only approximately one extra halving step.

C. Square-root growth — $O(\sqrt{n})$

Square-root complexity appears in algorithms where the amount of work grows proportionally to the square root of the input.

For example, when checking whether an integer n is prime, it is sufficient to test possible divisors up to $\sqrt{n}$.

This is because if:

$$n = a \times b$$

and both a and b were greater than $\sqrt{n}$, then:

$$a \times b > n$$

which is impossible.

Consider:

Plain text

```
Function IsPrime(n):

    if n < 2:
        return False

    for i from 2 to floor(sqrt(n)):
        if n mod i = 0:
            return False

    return True
```

Here:

Prime number: An integer greater than 1 that has exactly two positive divisors: 1 and itself.

Modulo (`mod`): An operation that returns the remainder after integer division.

For example:

$$15 \bmod 3 = 0$$

because 15 is divisible by 3.

For $n = 49$:

$$\sqrt{49} = 7$$

The possible divisors checked are:

```
Plain text

2, 3, 4, 5, 6, 7
```

The algorithm finds:

$$49 \bmod 7 = 0$$

Therefore, 49 is not prime.

The number of candidate divisors is bounded by a quantity proportional to $\sqrt{n}$.

Thus:

$$T(n) = O(\sqrt{n})$$

This counts division or modulo tests as constant-time operations.

D. Linear growth — $O(n)$

Linear complexity means the amount of work increases in proportion to the input size.

Example:

```
Plain text

Function PrintAll(A, n):

    for i from 0 to n-1:
        print A[i]
```

If the array contains 10 elements, the loop runs 10 times.

If the array contains 1,000 elements, the loop runs 1,000 times.

Therefore:

$$T(n) = O(n)$$

E. Linearithmic growth — $O(n \log n)$

Linearithmic complexity combines linear and logarithmic growth.

A simple example is:

```
Plain text
Function Example(n):
    for i from 1 to n:
        j ← 1
        while j < n:
            j ← 2*j
```

The outer loop executes n times.

For each outer iteration, the inner loop approximately doubles `j` until it reaches or exceeds n.

Therefore, the inner loop executes $O(\log n)$ times.

Total work:

$$T(n) = O(n) \times O(\log n)$$

$$\boxed{T(n) = O(n \log n)}$$

Efficient comparison-based sorting algorithms often have $O(n \log n)$ runtime.

F. Quadratic growth — $O(n^2)$

Quadratic complexity commonly occurs when one loop runs n times inside another loop that also runs n times.

Example:

```
Plain text
```

```
Function PrintPairs(n):

    for i from 1 to n:

        for j from 1 to n:

            print i, j
```

For $n = 3$, the output is:

Plain text

```
(1,1)
(1,2)
(1,3)

(2,1)
(2,2)
(2,3)

(3,1)
(3,2)
(3,3)
```

There are:

$$3 \times 3 = 9$$

executions of the inner statement.

In general:

$$n \times n = n^2$$

Therefore:

$$T(n) = O(n^2)$$

G. Exponential growth — $O(2^n)$

Exponential complexity occurs when computational work grows exponentially with the input size.

For example, generating all subsets of a set containing n elements involves:

$$2^n$$

subsets.

For a set:

$$S = \{A, B, C\}$$

the subsets are:

```
{}
{A}
{B}
{C}
{A,B}
{A,C}
{B,C}
{A,B,C}
```

There are eight subsets.

$$2^3 = 8$$

For $n = 20$:

$$2^{20} = 1,048,576$$

For $n = 30$:

$$2^{30} = 1,073,741,824$$

The number of subsets becomes extremely large even for moderate input sizes.

An algorithm that explicitly generates each subset needs at least $\Omega(2^n)$ work. With additional work per subset, its total runtime can be larger.

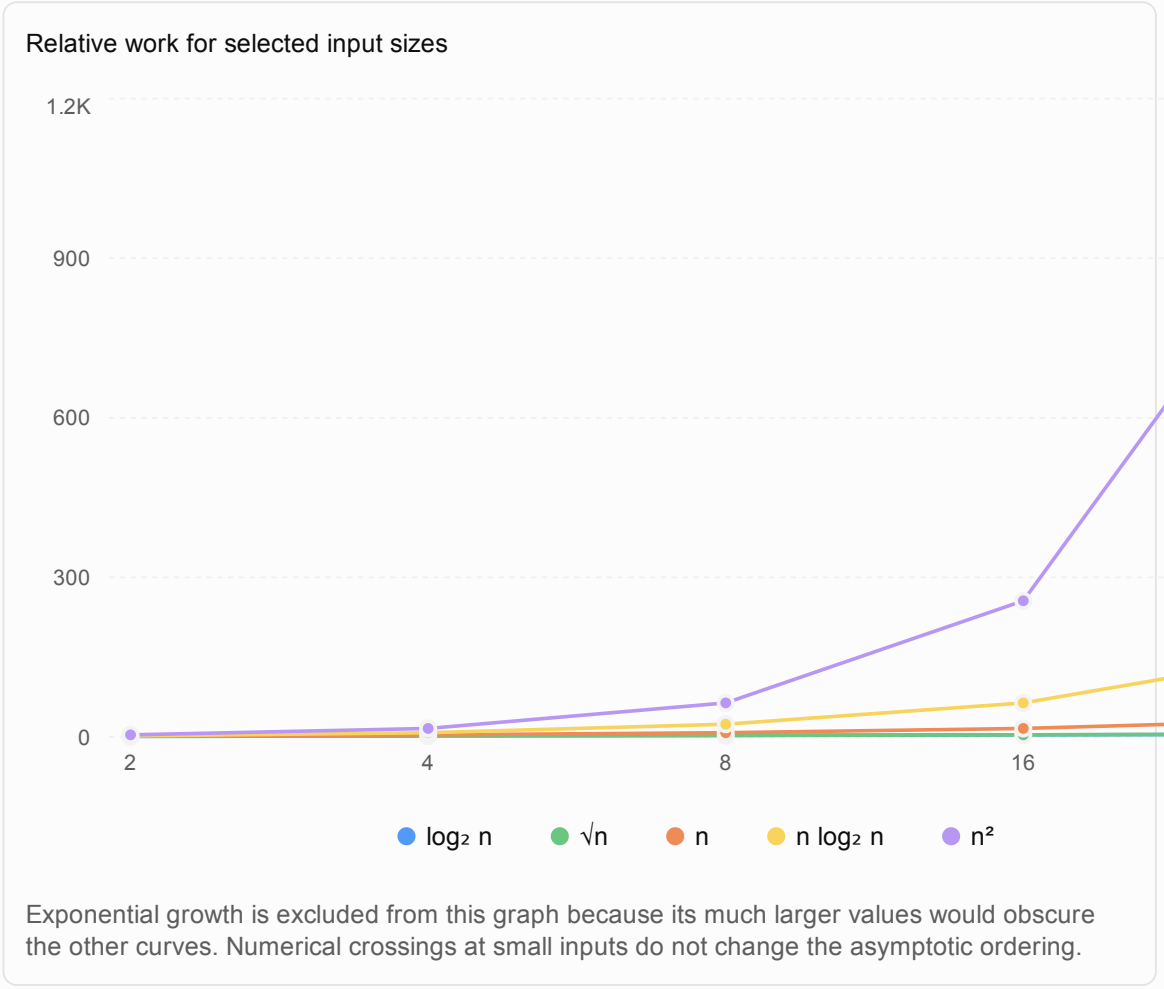
2.6 Comparing Common Growth Rates

For sufficiently large n, the following functions are ordered from slower to faster growth:

$$\boxed{\log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec 2^n}$$

The symbol $\prec$ means that the function on the left grows asymptotically strictly more slowly than the function on the right.

This ordering describes long-term growth, not necessarily the numerical ordering for every small input.



Understanding the ordering mathematically

Consider:

$$n < n^2$$

for $n > 1$.

If we divide both sides by n^2:

$$\frac{n}{n^2} = \frac{1}{n}$$

As n approaches infinity:

$$\lim_{n \to \infty} \frac{1}{n} = 0$$

This shows that linear growth becomes negligible compared with quadratic growth.

Similarly:

$$\lim_{n \to \infty} \frac{n^2}{2^n} = 0$$

This means that 2^n grows asymptotically faster than n^2.

2.7 Growth Rate and Practical Feasibility

Imagine a simplified computer that can perform 10^8 basic operations per second.

For an algorithm performing exactly n, n^2, or 2^n operations, the estimated runtime is:

$$\text{Time} = \frac{\text{Number of operations}}{10^8}$$

Input size	Linear n	Quadratic n^2	Exponential 2^n
20	0.0000002 s	0.000004 s	0.0105 s
30	0.0000003 s	0.000009 s	10.74 s
50	0.0000005 s	0.000025 s	About 130 days
100	0.000001 s	0.0001 s	About 4.0×10^{14} years

These are illustrative estimates based on a hypothetical constant operation rate. Actual programs have different costs per operation.

Nevertheless, the table demonstrates why growth rates matter.

An exponential algorithm can become impractical at input sizes that are easy for linear or quadratic algorithms.

Chapter 2 Summary

Asymptotic analysis studies how computational work grows with input size. It deliberately ignores constant multipliers and emphasizes the mathematical structure of growth.

Logarithmic, linear, and quadratic functions grow at very different rates. Exponential functions grow particularly quickly and often become impractical for large inputs.

The central benefit of asymptotic analysis is that it lets us compare algorithms without needing exact execution times.

Chapter 3: Big-O Notation

3.1 Introduction

Big-O notation is a mathematical notation used to express an asymptotic upper bound on a function.

In algorithm analysis, it is commonly used to describe an upper bound on the amount of work an algorithm requires as the input size increases.

For example:

$$T(n) = O(n^2)$$

means that the runtime function $T(n)$ does not grow faster than some constant multiple of n^2, once n is sufficiently large.

Big-O does not necessarily describe the exact growth rate. It describes an upper bound.

3.2 Formal Definition of Big-O

Let $f(n)$ and $g(n)$ be nonnegative functions for sufficiently large inputs.

We write:

$$\boxed{f(n) = O(g(n))}$$

if there exist positive constants c and N such that:

$$\boxed{f(n) \leq c\, g(n) \quad \text{for all } n \geq N}$$

Explanation of each symbol

Symbol	Meaning
$f(n)$	Function whose growth is being studied
$g(n)$	Function used as an upper-bound reference
O	Big-O notation
c	Positive constant multiplier
N	Input-size threshold
$n \geq N$	Inputs at least as large as the threshold
$\leq$	Less than or equal to

The definition says that, after some input size N, the function $f(n)$ is always at most c times $g(n)$.

The constants do not need to be the smallest possible constants. We only need to find values that satisfy the inequality.

3.3 Graphical Interpretation of Big-O

Consider:

$$f(n) = 3n^2 + 5n + 2$$

and:

$$g(n) = n^2$$

To show that:

$$f(n) = O(n^2)$$

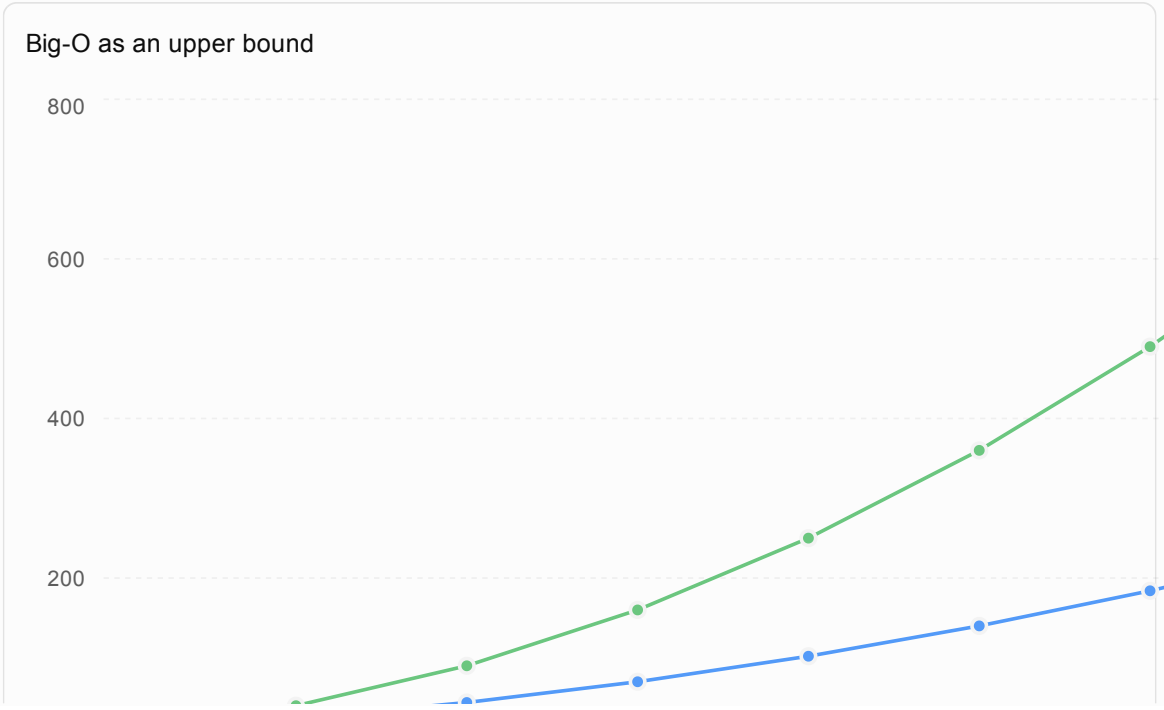
we must find constants c and N such that:

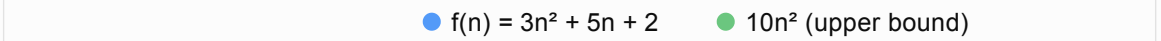
$$3n^2 + 5n + 2 \leq cn^2$$

for every $n \geq N$.

For example, we can choose:

$$c = 10, \quad N = 1$$





The function $10n^2$ is an upper bound for $3n^2 + 5n + 2$ for every integer $n \geq 1$.

The graph shows that the function $10n^2$ remains above or equal to $f(n)$.

This establishes that $f(n)$ belongs to $O(n^2)$.

3.4 Worked Proof: $3n^2 + 5n + 2 = O(n^2)$

We want to prove:

$$3n^2 + 5n + 2 = O(n^2)$$

Step 1: Write the required inequality

$$3n^2 + 5n + 2 \leq cn^2$$

Step 2: Assume $n \geq 1$

For $n \geq 1$:

$$n \leq n^2$$

Therefore:

$$5n \leq 5n^2$$

Also:

$$2 \leq 2n^2$$

Step 3: Substitute these upper bounds

$$3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2$$

Step 4: Combine the terms

$$3n^2 + 5n^2 + 2n^2 = 10n^2$$

Therefore:

$$3n^2 + 5n + 2 \leq 10n^2$$

for every $n \geq 1$.

Step 5: Identify the constants

$$c = 10$$

$$N = 1$$

Both satisfy the Big-O definition.

Hence:

$$\boxed{3n^2 + 5n + 2 = O(n^2)}$$

What is the purpose of this proof?

The proof mathematically establishes that the runtime grows no faster than a constant multiple of n^2 for sufficiently large inputs.

It also shows why the lower-order terms $5n$ and 2 do not change the quadratic asymptotic upper bound.

3.5 Understanding the Dominant Term

Consider the function:

$$T(n) = 7n^3 + 4n^2 + 10n + 5$$

The terms have different growth rates:

- $7n^3$: Cubic.
- $4n^2$: Quadratic.
- $10n$: Linear.
- 5: Constant.

As n increases, n^3 grows faster than the other terms.

Therefore, it is called the **dominant term**.

A dominant term is the term that determines the overall asymptotic growth rate of a sum of nonnegative terms.

We simplify:

$$7n^3 + 4n^2 + 10n + 5$$

to:

$$\boxed{O(n^3)}$$

Dry run using numerical values

n	$7n^3$	$4n^2$	$10n$	**Constant**
2	56	16	20	5
10	7,000	400	100	5
100	7,000,000	40,000	1,000	5
1,000	7,000,000,000	4,000,000	10,000	5

As n becomes larger, the cubic term accounts for an increasingly large proportion of the total.

Therefore, the lower-order terms can be omitted when expressing the asymptotic growth rate.

3.6 Big-O Is an Upper Bound, Not Necessarily a Tight Bound

Consider:

$$f(n) = n$$

We can correctly state:

$$f(n) = O(n)$$

But we can also state:

$$f(n) = O(n^2)$$

and:

$$f(n) = O(n^3)$$

These are all mathematically valid upper bounds.

For example, when $n \geq 1$:

$$n \leq n^2$$

and:

$$n \leq n^3$$

However, $O(n)$ is the most informative of these three upper bounds.

A **tight asymptotic bound** describes the actual order of growth up to constant factors and is expressed using Theta notation:

$$f(n) = \Theta(n)$$

Thus, when describing algorithms, we normally prefer the tightest useful bound that we can establish.

3.7 Simplifying Big-O Expressions

Rule 1: Ignore multiplicative constants

Consider:

$$T(n) = 7n^3$$

The constant 7 does not affect the asymptotic growth.

Therefore:

$$\boxed{7n^3 = O(n^3)}$$

Similarly:

$$100n^2 = O(n^2)$$

and:

$$0.5n = O(n)$$

Rule 2: Keep the highest-degree polynomial term

Consider:

$$T(n) = 4n^4 + 8n^2 + 12$$

The dominant term is:

$$4n^4$$

After ignoring its constant multiplier:

$$\boxed{T(n) = O(n^4)}$$

Rule 3: Polynomial functions grow more slowly than exponential functions

For every fixed $a > 0$ and every constant $b > 1$:

$$n^a = o(b^n)$$

For example:

$$n^5 = O(2^n)$$

and:

$$n^{100} = O(1.1^n)$$

Although the exponential functions may initially be smaller, they eventually dominate as n becomes sufficiently large.

Rule 4: Logarithmic powers grow more slowly than positive polynomial powers

For fixed $a > 0$ and $b > 0$:

$$(\log n)^a = o(n^b)$$

For example:

$$(\log n)^3 = O(\sqrt{n})$$

and:

$$n \log n = O(n^2)$$

Rule 5: Discard smaller terms

Consider:

$$T(n) = n^2 + n$$

Since n^2 dominates n:

$$\boxed{T(n) = O(n^2)}$$

Consider another expression:

$$T(n) = 2^n + n^9$$

Since exponential growth eventually dominates polynomial growth:

$$\boxed{T(n) = O(2^n)}$$

Note that 2^n means 2 raised to the power n, whereas $2n$ means 2 multiplied by n. They represent completely different growth rates.

3.8 Logarithms in Algorithm Analysis

A **logarithm** is the inverse operation of exponentiation.

For example:

$$2^3 = 8$$

Therefore:

$$\log_2 8 = 3$$

Similarly:

$$2^{10} = 1024$$

Therefore:

$$\log_2 1024 = 10$$

In general:

$$b^x = n$$

is equivalent to:

$$\log_b n = x$$

Here, b is called the **base** of the logarithm.

Why is the logarithm base often omitted in Big-O?

The change-of-base formula states:

$$\log_a n = \frac{\log_b n}{\log_b a}$$

The denominator is a constant when a and b are fixed valid bases.

Therefore, logarithms with different fixed bases differ only by constant multipliers.

For example:

$$\log_2 n = \frac{\log_3 n}{\log_3 2}$$

Since:

$$\frac{1}{\log_3 2}$$

is constant, the asymptotic growth remains the same.

Hence:

$$O(\log_2 n) = O(\log_3 n)$$

In algorithm analysis, we commonly write:

$$\boxed{O(\log n)}$$

without specifying the base.

3.9 Advantages of Big-O Notation

Big-O notation provides several important benefits.

1. Simplifies mathematical expressions

Instead of writing:

$$3n^2 + 5n + 2$$

we can write:

$$O(n^2)$$

This makes the growth category easier to understand.

2. Supports comparison of algorithms

Suppose:

$$T_A(n) = O(n)$$

and:

$$T_B(n) = O(n^2)$$

If these are also tight bounds, Algorithm A has a better asymptotic growth rate.

3. Reduces dependence on hardware details

Big-O expresses mathematical growth rather than exact processor time.

4. Helps predict scalability

We can estimate how the work changes as the input size increases.

5. Makes mathematical analysis easier

Complicated expressions can often be simplified by retaining only their dominant terms.

3.10 Limitations of Big-O Notation

Big-O is useful, but it does not tell us everything about an algorithm.

Limitation 1: It ignores constant factors

Consider:

$$T_A(n) = 100n$$

$$T_B(n) = n^2$$

For $n = 10$:

$$T_A(10) = 1000$$

$$T_B(10) = 100$$

Although Algorithm A has linear growth, it performs more work for this input size.

The functions become equal when:

$$100n = n^2$$

For positive n, dividing both sides by n:

$$100 = n$$

At:

$$n = 100$$

both functions produce 10,000 operations.

For $n > 100$, the quadratic function becomes larger.

This shows that an algorithm with a better asymptotic growth rate is not necessarily faster for every small input.

Limitation 2: It does not give exact execution time

An $O(n)$ algorithm might take 1 millisecond or 10 seconds for a particular input, depending on its constant factors, the hardware, and the implementation.

Limitation 3: Big-O is not automatically worst-case complexity

Big-O describes an asymptotic upper bound on whichever runtime function is being analyzed.

The function could represent:

- Worst-case runtime.

- Average-case runtime.
- Best-case runtime.

For example, an algorithm may have an average-case runtime of $O(n)$ and a worst-case runtime of $O(n^2)$.

Therefore, the type of runtime being analyzed should be stated explicitly.

Chapter 3 Summary

Big-O notation expresses an asymptotic upper bound. It allows us to disregard constant multipliers and lower-order terms when studying long-term growth.

It makes algorithm comparisons easier, but it does not describe exact execution times or preserve all practical performance information.

Chapter 4: Applying Big-O Notation to Algorithms

4.1 Introduction

Understanding the mathematical definition of Big-O is important, but students must also learn how to apply it to actual algorithms.

To determine an algorithm's time complexity, we generally examine:

1. The operations performed by the algorithm.
2. How often each operation executes.
3. The computational cost of each operation.
4. How these costs combine as the input size increases.

A useful distinction is between **the number of executions of an instruction** and **the cost of one execution**.

For example, an instruction that executes n times and costs $O(n)$ per execution can contribute $O(n^2)$ total work.

4.2 Rules for Analyzing Algorithm Runtime

Rule 1: Sequential operations are added

Consider:

Plain text

```
x ← 10
y ← 20
z ← x + y
```

Under the standard constant-time arithmetic model, each instruction takes $O(1)$.

Therefore:

$$T(n) = O(1) + O(1) + O(1)$$

$$T(n) = O(3)$$

After ignoring the constant:

$$\boxed{T(n) = O(1)}$$

Rule 2: Repeated operations are multiplied by their execution count

Consider:

Plain text

```
for i from 1 to n:
    print(i)
```

The print statement executes n times.

Assuming a constant cost for each basic output operation:

$$T(n) = n \times O(1)$$

Therefore:

$$\boxed{T(n) = O(n)}$$

For actual text output, printing larger numbers may require more characters, so this simplified model abstracts away that additional cost.

Rule 3: Nested loops require analyzing the total inner-loop executions

Consider:

Plain text

```
for i from 1 to n:
    for j from 1 to n:
```

```
        x ← x + 1
```

The outer loop executes n times.

For each outer iteration, the inner loop executes n times.

Therefore:

$$T(n) = n \times n \times O(1)$$

$$\boxed{T(n) = O(n^2)}$$

Rule 4: Consecutive loops are added

Consider:

```
Plain text
for i from 1 to n:
    print(i)

for j from 1 to n:
    print(j)
```

The first loop executes n times.

The second loop also executes n times.

Therefore:

$$T(n) = O(n) + O(n)$$

$$T(n) = O(2n)$$

$$\boxed{T(n) = O(n)}$$

Two consecutive linear loops do not produce quadratic complexity.

Rule 5: Keep the dominant term

Consider:

```
Plain text

for i from 1 to n:
    print(i)

for i from 1 to n:
    for j from 1 to n:
        print(i, j)
```

The first loop requires:

$$O(n)$$

The nested loops require:

$$O(n^2)$$

Therefore:

$$T(n) = O(n) + O(n^2)$$

The dominant term is n^2.

Thus:

$$\boxed{T(n) = O(n^2)}$$

4.3 Detailed Runtime Analysis of FibList

Consider the Fibonacci algorithm:

```
Plain text

Function FibList(n)

    create an array F[0 ... n]

    F[0] ← 0
    F[1] ← 1

    for i from 2 to n:

        F[i] ← F[i-1] + F[i-2]

    return F[n]
```

The runtime depends on the cost model used for storing and adding Fibonacci numbers.

We will examine two models.

Model A: Constant-Time Arithmetic

In introductory algorithm analysis, we often assume that basic integer arithmetic takes constant time.

Under this assumption:

Instruction	Number of executions	Cost per execution	Total cost
Array creation and initialization	1	$O(n)$	$O(n)$
`F[0] ← 0`	1	$O(1)$	$O(1)$
`F[1] ← 1`	1	$O(1)$	$O(1)$
Loop body	$n-1$	$O(1)$	$O(n)$
`return F[n]`	1	$O(1)$	$O(1)$

The total runtime is:

$$T(n) = O(n) + O(1) + O(1) + O(n) + O(1)$$

Combining the terms:

$$T(n) = O(n) + O(n) + O(1)$$

Therefore:

$$\boxed{T(n) = O(n)}$$

The algorithm has linear time complexity under the constant-time arithmetic assumption.

Its array also requires $O(n)$ storage locations.

Model B: Accounting for Large Integer Arithmetic

The previous model assumes that adding any two integers takes the same amount of time.

However, Fibonacci numbers become larger as their index increases.

For example:

$$F(10) = 55$$

$$F(20) = 6765$$

$$F(50) = 12,586,269,025$$

$$F(100) = 354224848179261915075$$

Larger integers require more bits to represent.

A **bit** is a binary digit that can hold either 0 or 1.

An integer containing b bits can require $O(b)$ time to add to another integer of comparable size using ordinary digit-by-digit binary addition.

Fibonacci numbers grow exponentially in their index, so the number of bits required to represent $F(i)$ grows proportionally to i.

In particular:

$$\text{Bits in } F(i) = \Theta(i)$$

for sufficiently large i.

Therefore, calculating:

```
Plain text
F[i] ← F[i-1] + F[i-2]
```

requires:

$$O(i)$$

bit operations using standard linear-time integer addition.

Since:

$$i \leq n$$

each addition also has the coarser upper bound:

$$O(n)$$

The loop executes $O(n)$ times.

Using this upper bound:

$$O(n) \times O(n) = O(n^2)$$

The total runtime is therefore bounded by:

$$T(n) = O(n) + O(1) + O(1) + O(n^2) + O(1)$$

Hence:

$$\boxed{T(n) = O(n^2)}$$

under the bit-operation model with ordinary integer addition.

This is an important result: **the same algorithm can have different time-complexity expressions depending on the computational model.**

4.4 Deriving the Quadratic Runtime More Precisely

Instead of assuming that every addition costs $O(n)$, consider the cost of addition during each loop iteration.

At iteration i, the addition involves Fibonacci numbers whose bit lengths are proportional to i.

Thus, the work performed during iteration i is:

$$\Theta(i)$$

The total work is approximately:

$$T(n) = \sum_{i=2}^{n} \Theta(i)$$

The symbol $\sum$ is called **sigma notation** and represents a sum of values.

For example:

$$\sum_{i=1}^{5} i = 1 + 2 + 3 + 4 + 5$$

$$= 15$$

For the Fibonacci algorithm:

$$\sum_{i=2}^{n} i = 2 + 3 + 4 + \cdots + n$$

The formula for the sum of the first n positive integers is:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Therefore:

$$2 + 3 + \cdots + n = \frac{n(n+1)}{2} - 1$$

Expanding:

$$= \frac{n^2 + n}{2} - 1$$

The dominant term is:

$$\frac{n^2}{2}$$

Hence:

$$\boxed{T(n) = \Theta(n^2)}$$

for the standard bit-operation model with ordinary integer addition.

4.5 Numerical Dry Run of the Runtime Calculation

Consider:

$$n = 6$$

The loop executes at indices:

$$i = 2, 3, 4, 5, 6$$

The following table illustrates a simplified accounting system in which the work at iteration i is represented by i units.

Iteration	Index i	Illustrative work units	Cumulative units
1	2	2	2
2	3	3	5
3	4	4	9
4	5	5	14
5	6	6	20

Total:

$$2 + 3 + 4 + 5 + 6 = 20$$

Using the summation formula:

$$\frac{6(6+1)}{2} - 1$$

$$= \frac{42}{2} - 1$$

$$= 21 - 1$$

$$= 20$$

The value 20 is an illustrative cost-unit total, not an exact count of processor instructions or bit operations.

The important observation is that the sum grows quadratically:

$$\sum_{i=2}^{n} i = \Theta(n^2)$$

4.6 Space Complexity of FibList

Space complexity describes how memory requirements grow with the input size.

The algorithm creates:

```
Plain text
F[0 ... n]
```

The array contains:

$$n+1$$

elements.

Under a fixed-size-element model:

$$S(n) = O(n+1)$$

Therefore:

$$\boxed{S(n) = O(n)}$$

However, when Fibonacci numbers are stored as arbitrary-precision integers, the total memory required to store all array values is larger.

Since the bit length of $F(i)$ is $\Theta(i)$, the total number of stored bits is:

$$\sum_{i=1}^{n} \Theta(i)$$

Therefore:

$$\boxed{S(n) = \Theta(n^2) \text{ bits}}$$

This distinction reinforces the importance of clearly stating the computational model.

4.7 Common Algorithm Dry Runs

Example 1: Linear Search

Problem: Find whether a target element exists in an array.

```
Plain text
```

```
Function LinearSearch(A, n, target):

    for i from 0 to n-1:

        if A[i] = target:
            return i

    return -1
```

Here, `-1` indicates that the target was not found.

Dry run

Input:

```
Plain text

A = [9, 4, 12, 7, 15]
target = 7
```

Iteration	Index	Element	Comparison	Result
1	0	9	9 = 7?	False
2	1	4	4 = 7?	False
3	2	12	12 = 7?	False
4	3	7	7 = 7?	True

The algorithm returns index 3.

The target is found after four comparisons.

If the target is not present, the algorithm examines all n elements.

Therefore, the worst-case runtime is:

$$\boxed{O(n)}$$

Example 2: Triangular Nested Loop

Consider:

```
Plain text

Function Example(n):

    for i from 1 to n:
```

```
        for j from 1 to i:
            print(i, j)
```

This is different from a nested loop in which both loops always execute n times.

Here, the number of inner-loop executions depends on `i`.

Dry run for $n = 4$

Outer iteration i	Inner-loop values	Number of executions
1	1	1
2	1, 2	2
3	1, 2, 3	3
4	1, 2, 3, 4	4

Total:

$$1 + 2 + 3 + 4 = 10$$

In general:

$$T(n) = 1 + 2 + \cdots + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2 + n}{2}$$

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

This demonstrates that nested-loop analysis sometimes requires summation rather than simply multiplying the loop bounds.

Example 3: Logarithmic Loop

Consider:

```
Plain text
Function Example(n):
    i ← 1
    while i < n:
        i ← 2*i
```

Dry run for $n = 20$

Iteration	Value before doubling	Value after doubling
1	1	2
2	2	4
3	4	8
4	8	16
5	16	32

The loop stops because:

$$32 \geq 20$$

The number of iterations is:

$$\lceil \log_2 20 \rceil = 5$$

The symbol $\lceil x \rceil$ is the **ceiling function**, which rounds a number upward to the nearest integer.

For example:

$$\lceil 4.32 \rceil = 5$$

Therefore:

$$\boxed{T(n) = O(\log n)}$$

Chapter 4 Summary

The runtime of an algorithm depends on both how many times its operations execute and how expensive those operations are.

Sequential operations are added, repeated operations are multiplied by their execution counts, and dominant terms determine the asymptotic growth.

Careful analysis also requires selecting a computational model. The FibList example illustrates how constant-time integer arithmetic and variable-size integer arithmetic lead to different complexity results.

Chapter 5: Other Asymptotic Notations

5.1 Introduction

Big-O is one of the most widely used notations in algorithm analysis. However, it is not the only notation used to describe the growth of functions.

Other important notations include:

- Big-Omega: Ω
- Big-Theta: Θ
- Little-o: o

Each notation describes a different mathematical relationship between two functions.

Before studying these notations, it is useful to distinguish three concepts.

Upper bound: A function that grows at least as fast as the function being analyzed, up to constant factors.

Lower bound: A function that grows no faster than the function being analyzed, up to constant factors.

Tight bound: A function that grows at the same asymptotic rate as the function being analyzed.

5.2 Big-Omega Notation — Ω

Big-Omega notation is used to describe an **asymptotic lower bound**.

Formal Definition

We write:

$$\boxed{f(n) = \Omega(g(n))}$$

if there exist positive constants c and N such that:

$$\boxed{f(n) \geq c\, g(n)}$$

for all:

$$n \geq N$$

The definition means that $f(n)$ grows at least as fast as some positive constant multiple of $g(n)$, for sufficiently large values of n.

Example

Consider:

$$f(n) = 3n^2 + 5n + 2$$

We want to prove:

$$f(n) = \Omega(n^2)$$

According to the definition, we must show that:

$$3n^2 + 5n + 2 \geq cn^2$$

For every $n \geq 1$:

$$3n^2 + 5n + 2 \geq 3n^2$$

Therefore, choose:

$$c = 3$$

and:

$$N = 1$$

The required inequality is satisfied.

Thus:

$$\boxed{3n^2 + 5n + 2 = \Omega(n^2)}$$

Interpretation

The function $3n^2 + 5n + 2$ cannot grow asymptotically more slowly than n^2.

It has a quadratic lower bound.

5.3 Big-Theta Notation — Θ

Big-Theta notation is used to describe a **tight asymptotic bound**.

A function has a tight bound when it is bounded both above and below by constant multiples of another function.

Formal Definition

We write:

$$\boxed{f(n) = \Theta(g(n))}$$

if there exist positive constants c_1, c_2, and N such that:

$$\boxed{c_1 g(n) \leq f(n) \leq c_2 g(n)}$$

for all:

$$n \geq N$$

In simple words, $f(n)$ and $g(n)$ have the same asymptotic growth rate.

Relationship Between Big-O, Big-Omega, and Big-Theta

A function is $\Theta(g(n))$ if it is both $O(g(n))$ and $\Omega(g(n))$.

Mathematically:

$$\boxed{f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))}$$

The symbol $\iff$ means *if and only if*.

Worked Example

Consider:

$$f(n) = 3n^2 + 5n + 2$$

We already established that:

$$3n^2 + 5n + 2 \leq 10n^2$$

for $n \geq 1$.

Therefore:

$$f(n) = O(n^2)$$

We also established:

$$3n^2 + 5n + 2 \geq 3n^2$$

Therefore:

$$f(n) = \Omega(n^2)$$

Combining both inequalities:

$$3n^2 \leq 3n^2 + 5n + 2 \leq 10n^2$$

Here:

$$c_1 = 3, \quad c_2 = 10, \quad N = 1$$

Thus:

$$\boxed{3n^2 + 5n + 2 = \Theta(n^2)}$$

This is a stronger statement than merely saying the function is $O(n^2)$.

It confirms that its growth rate is quadratic up to constant factors.

5.4 Little-o Notation — o

Little-o notation describes a **strict asymptotic upper bound**.

It indicates that one function grows asymptotically more slowly than another.

Formal Definition

We write:

$$\boxed{f(n) = o(g(n))}$$

if:

$$\boxed{\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0}$$

assuming the ratio is defined for sufficiently large n.

Explanation

A **limit** describes the value that a mathematical expression approaches as its input changes.

In this definition, we examine the ratio between the two functions.

If the ratio approaches zero, the numerator grows asymptotically more slowly than the denominator.

Example 1: Linear versus quadratic growth

Let:

$$f(n) = n$$

$$g(n) = n^2$$

Calculate the ratio:

$$\frac{f(n)}{g(n)} = \frac{n}{n^2}$$

Simplifying:

$$= \frac{1}{n}$$

Taking the limit:

$$\lim_{n \to \infty} \frac{1}{n} = 0$$

Therefore:

$$\boxed{n = o(n^2)}$$

Example 2: Quadratic versus cubic growth

Consider:

$$f(n) = n^2$$

$$g(n) = n^3$$

Then:

$$\frac{f(n)}{g(n)} = \frac{n^2}{n^3}$$

$$= \frac{1}{n}$$

As $n \to \infty$:

$$\frac{1}{n} \to 0$$

Therefore:

$$\boxed{n^2 = o(n^3)}$$

Example 3: Why $3n^2$ is not $o(n^2)$

Consider:

$$f(n) = 3n^2$$

$$g(n) = n^2$$

The ratio is:

$$\frac{3n^2}{n^2} = 3$$

Therefore:

$$\lim_{n \to \infty} \frac{3n^2}{n^2} = 3$$

The limit is not zero.

Hence:

$$\boxed{3n^2 \neq o(n^2)}$$

However:

$$3n^2 = O(n^2)$$

and:

$$3n^2 = \Theta(n^2)$$

This demonstrates the difference between Big-O and little-o.

5.5 Comparison of Asymptotic Notations

Notation	Meaning	Mathematical relationship
$O(g(n))$	Asymptotic upper bound	$f(n) \leq cg(n)$ eventually
$\Omega(g(n))$	Asymptotic lower bound	$f(n) \geq cg(n)$ eventually
$\Theta(g(n))$	Tight asymptotic bound	$c_1 g(n) \leq f(n) \leq c_2 g(n)$ eventually
$o(g(n))$	Strictly slower asymptotic growth	$f(n)/g(n) \to 0$

The word *eventually* means that the relationship holds for every input size beyond some fixed threshold.

5.6 Understanding the Difference with One Function

Consider:

$$f(n) = 5n + 10$$

We can make the following statements:

$$f(n) = O(n)$$

because $f(n)$ grows no faster than a constant multiple of n.

$$f(n) = \Omega(n)$$

because $f(n)$ grows at least as fast as a constant multiple of n.

$$f(n) = \Theta(n)$$

because both the upper and lower bounds are linear.

We can also write:

$$f(n) = o(n^2)$$

because:

$$\lim_{n \to \infty} \frac{5n + 10}{n^2} = \lim_{n \to \infty} \left(\frac{5}{n} + \frac{10}{n^2} \right) = 0$$

But we cannot write:

$$f(n) = o(n)$$

because:

$$\lim_{n \to \infty} \frac{5n + 10}{n} = 5$$

The limit is 5 rather than 0.

5.7 Asymptotic Notation and Best-, Average-, and Worst-Case Analysis

Students frequently confuse Big-O with worst-case analysis and Big-Omega with best-case analysis.

These concepts are related, but they are not identical.

Best-case runtime

The minimum amount of work required by an algorithm among all inputs of a particular size.

Worst-case runtime

The maximum amount of work required among all inputs of a particular size.

Average-case runtime

The expected amount of work over inputs of a particular size, under a specified probability distribution.

Example: Linear Search

Suppose the array is:

```
Plain text
A = [5, 8, 12, 20, 25]
```

If the target is 5, the algorithm finds it immediately.

Number of comparisons:

$$1$$

If the target is 25, the algorithm performs five comparisons.

If the target is absent, it also performs five comparisons.

For an array of n elements:

Case	Number of comparisons	Complexity
Best case	1	$\Theta(1)$
Average case, uniformly located successful target	$(n+1)/2$	$\Theta(n)$
Worst case	n	$\Theta(n)$

The average-case expression assumes that the target is present and equally likely to occupy any array position.

Big-O, Big-Omega, and Big-Theta can each be used to describe bounds on the best-case, average-case, or worst-case runtime functions.

For example, linear search has:

$$T_{\text{worst}}(n) = \Theta(n)$$

and:

$$T_{\text{best}}(n) = \Theta(1)$$

Chapter 5 Summary

Big-O describes an asymptotic upper bound, Big-Omega describes a lower bound, and Big-Theta describes a tight bound.

Little-o indicates that one function grows strictly more slowly than another in the asymptotic sense.

These mathematical tools allow us to describe and compare algorithm growth precisely.

Chapter 6: Introduction to Algorithm Design Techniques

6.1 What Is Algorithm Design?

Algorithm design is the process of developing a systematic, correct, and efficient procedure for solving a computational problem.

Algorithm design involves more than writing program instructions.

It requires understanding the problem, selecting an appropriate strategy, proving or reasoning about correctness, and analyzing the computational resources required.

For example, imagine that a student needs to search for a name in a list of one million names.

Several approaches may be possible:

- Examine every name one by one.
- Sort the names and then use binary search.
- Store the names in a suitable data structure for repeated searches.

Each approach has different advantages, limitations, and computational costs.

The challenge is to select an appropriate method for the problem and its requirements.

6.2 Why Is Algorithm Design Difficult?

Algorithm design can be difficult because computational problems have different structures.

An approach that is effective for one problem may not work for another.

For example:

Searching problems: Require locating particular data.

Sorting problems: Require arranging elements in a specified order.

Shortest-path problems: Require finding paths through a network that minimize a chosen cost.

Scheduling problems: Require assigning tasks to time intervals or resources while satisfying constraints.

Optimization problems: Require finding a solution that minimizes or maximizes an objective function.

An **objective function** is the mathematical quantity that an optimization problem attempts to minimize or maximize.

A **constraint** is a condition that a valid solution must satisfy.

There is no single universal procedure that automatically produces an efficient algorithm for every computational problem.

Instead, algorithm designers use mathematical reasoning, common algorithmic techniques, practical experience, and sometimes new insights.

6.3 The Algorithm Designer's Toolbox

A **toolbox** in algorithm design refers to a collection of general strategies that can be applied to many different problems.

Three important algorithm design techniques are:

1. Greedy algorithms.
2. Divide and conquer.
3. Dynamic programming.

Each technique provides a different way to organize the solution of a problem.

6.4 Greedy Algorithms

A **greedy algorithm** constructs a solution by repeatedly making a choice that appears best at the current step.

The main idea is to make a locally optimal choice.

A **locally optimal choice** is the best choice according to the information available at the current step.

A **globally optimal solution** is the best possible complete solution to the original problem.

A greedy algorithm does not automatically produce a globally optimal solution. Its correctness depends on the particular problem and the greedy rule used.

Example: Activity Selection

Suppose a student wants to attend the maximum number of non-overlapping activities.

Each activity has a starting time and a finishing time.

Activity	Start	Finish
A	1	3
B	2	5
C	3	4
D	4	7
E	6	8
F	7	9

The objective is to select the largest possible number of activities without time conflicts.

A greedy strategy is to repeatedly choose the compatible activity that finishes earliest.

Algorithm

Plain text

```
Function ActivitySelection(activities):

    sort activities by finishing time

    selected ← empty list

    lastFinish ← negative infinity

    for each activity in sorted activities:

        if activity.start >= lastFinish:

            add activity to selected

            lastFinish ← activity.finish

    return selected
```

Dry Run

After sorting by finishing time, the order is:

Plain text

```
A, C, B, D, E, F
```

Activity	Start	Finish	Decision	Reason
A	1	3	Select	First activity
C	3	4	Select	Starts when A finishes
B	2	5	Reject	Overlaps with C
D	4	7	Select	Starts when C finishes
E	6	8	Reject	Overlaps with D
F	7	9	Select	Starts when D finishes

Selected activities:

$$\boxed{\{A, C, D, F\}}$$

Total selected activities:

$$4$$

The earliest-finish-time strategy is optimal for this standard activity-selection problem.

The reason is that selecting an activity that finishes earlier leaves at least as much remaining time for later activities as selecting a compatible activity that finishes later.

Time Complexity

Sorting requires:

$$O(n \log n)$$

The selection loop requires:

$$O(n)$$

Therefore:

$$T(n) = O(n \log n) + O(n)$$

$$\boxed{T(n) = O(n \log n)}$$

6.5 Divide and Conquer

Divide and conquer is an algorithm design technique that solves a large problem by dividing it into smaller subproblems.

It generally consists of three steps.

Divide: Split the original problem into smaller subproblems.

Conquer: Solve the smaller subproblems, often recursively.

Combine: Combine the solutions to obtain the solution of the original problem.

What Is Recursion?

Recursion is a programming technique in which a function calls itself to solve smaller instances of the same problem.

A recursive algorithm normally contains:

Base case: A condition that stops further recursive calls.

Recursive case: An instruction that calls the function on a smaller problem.

Without a suitable base case or progress toward it, recursion may continue indefinitely.

Example: Binary Search

Binary search finds a target element in a sorted array by repeatedly reducing the search interval by approximately half.

Consider:

Plain text

```
A = [3, 7, 11, 15, 19, 23, 27, 31]
```

We want to find:

Algorithm

Plain text

```
Function BinarySearch(A, target):
```

```
low ← 0
high ← length(A) - 1

while low <= high:

    mid ← floor((low + high) / 2)

    if A[mid] = target:
        return mid

    else if A[mid] < target:
        low ← mid + 1

    else:
        high ← mid - 1
```

Explanation of Terms

Low: The index of the first element in the current search interval.

High: The index of the last element in the current search interval.

Mid: The index of the middle element of the current interval.

Sorted array: An array whose elements are arranged in a known order, such as ascending order.

Dry Run

Target:

$$23$$

Initial values:

$$low = 0, \quad high = 7$$

Iteration 1

Calculate:

$$mid = \left\lfloor \frac{0 + 7}{2} \right\rfloor$$

$$mid = 3$$

Array element:

$$A[3] = 15$$

Since:

$$15 < 23$$

discard the left portion, including index 3.

Update:

$$low = 4$$

Iteration 2

Now:

$$low = 4, \quad high = 7$$

Calculate:

$$mid = \left\lfloor \frac{4 + 7}{2} \right\rfloor$$

$$mid = 5$$

The array element is:

$$A[5] = 23$$

The target is found.

Return:

$$\boxed{5}$$

Dry Run Table

Iteration	Low	High	Mid	`A[mid]`	Action
1	0	7	3	15	Search right half
2	4	7	5	23	Target found

Runtime Analysis

Each iteration reduces the number of remaining elements by approximately half.

After k iterations, the number of candidates is approximately:

$$\frac{n}{2^k}$$

The process stops when the number of candidates is at most 1.

Thus:

$$\frac{n}{2^k} \leq 1$$

Multiplying both sides by 2^k:

$$n \leq 2^k$$

Taking logarithms:

$$\log_2 n \leq k$$

Therefore, the number of iterations required is logarithmic.

The worst-case time complexity is:

$$\boxed{O(\log n)}$$

An iterative binary search requires $O(1)$ additional space.

Binary search is an example of repeatedly reducing a problem to one smaller subproblem. Unlike some divide-and-conquer algorithms, it does not need a substantial combine step.

6.6 Dynamic Programming

Dynamic programming is an algorithm design technique used to solve problems by combining solutions to smaller, related subproblems.

It is especially useful when the same subproblems occur repeatedly.

Two important properties are:

Overlapping subproblems: The algorithm needs to solve the same smaller problems multiple times.

Optimal substructure: An optimal solution to the overall problem can be constructed from optimal solutions to suitable subproblems.

Not every dynamic programming problem is an optimization problem. The technique can also be used to count possibilities or calculate values such as Fibonacci numbers.

Example: Fibonacci Numbers

Recall:

$$F(n) = F(n-1) + F(n-2)$$

A direct recursive algorithm is:

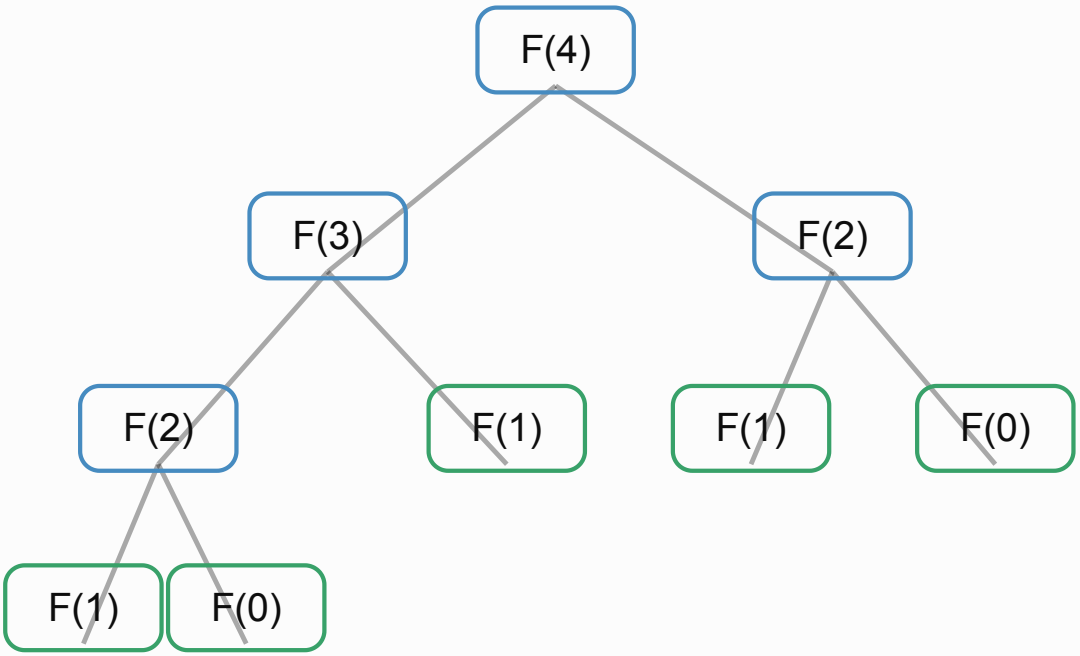
```
Plain text
Function FibRecursive(n):

    if n <= 1:
        return n

    return FibRecursive(n-1) + FibRecursive(n-2)
```

Recursive Dry Run for $F(4)$

Complete recursive call tree



Each box represents one function call. The leaf calls reach the base cases $F(0)$ or $F(1)$.

The recursive tree repeats the calculation of $F(2)$.

For larger values of n, many more repeated calls occur.

The recurrence describing the number of recursive calls is:

$$C(n) = C(n-1) + C(n-2) + 1$$

for $n \geq 2$, with:

$$C(0) = C(1) = 1$$

The extra 1 counts the current function call.

This count grows exponentially, specifically at rate $\Theta(\varphi^n)$, where:

$$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Therefore, the simple recursive Fibonacci algorithm has exponential time complexity under the constant-time arithmetic model.

Improving Fibonacci Using Dynamic Programming

Instead of recalculating the same values, we can store them.

Plain text

```
Function FibDP(n):

    if n <= 1:
        return n

    create an array F[0 ... n]

    F[0] ← 0
    F[1] ← 1

    for i from 2 to n:
        F[i] ← F[i-1] + F[i-2]

    return F[n]
```

This is an example of **bottom-up dynamic programming**.

Bottom-up means that the algorithm starts with the smallest subproblems and gradually builds solutions for larger subproblems.

Under constant-time arithmetic:

$$\boxed{\text{Time complexity} = O(n)}$$

$$\boxed{\text{Space complexity} = O(n)}$$

Accounting for arbitrary-precision Fibonacci integers and ordinary addition gives the larger bit-complexity bounds discussed previously.

Two Common Dynamic Programming Approaches

Memoization (top-down): Uses recursion and stores results of previously solved subproblems so they are not recalculated.

Tabulation (bottom-up): Solves smaller subproblems in a suitable order and stores their results, usually in a table or array.

Both approaches reduce repeated work when the problem has overlapping subproblems.

6.7 Levels of Algorithm Design

Algorithm design can be understood through several levels of sophistication.

Level 1: Naive Algorithm

A **naive algorithm** uses a straightforward approach based directly on the problem definition.

It often requires little design effort but may be inefficient.

For example, to find a target in an unsorted array, we can examine each element sequentially.

This produces linear search.

A naive approach is not necessarily incorrect. It is often useful as a first solution and as a reference for evaluating improved methods.

Level 2: Algorithm Using Standard Techniques

At this level, a problem is solved using a well-known algorithm design technique.

Examples include:

- Greedy algorithms.
- Divide and conquer.
- Dynamic programming.

Using standard techniques helps transform a straightforward solution into a more systematic and efficient algorithm.

For example, when an array is already sorted, binary search is usually preferable to linear search for a single search operation.

Level 3: Optimized Algorithm

An **optimized algorithm** improves an existing solution by reducing unnecessary computation, reducing memory usage, or improving how operations are organized.

For example, consider the Fibonacci algorithm that stores every Fibonacci number in an array.

If we only need $F(n)$, storing the entire sequence is unnecessary.

Each new Fibonacci number depends only on the previous two numbers.

Therefore, we can use just two variables.

Space-Optimized Fibonacci Algorithm

Plain text

```
Function FibOptimized(n):

    if n <= 1:
        return n

    a ← 0
    b ← 1

    for i from 2 to n:

        c ← a + b

        a ← b
        b ← c

    return b
```

Dry Run for $n = 6$

Initial values:

$$a = 0, \quad b = 1$$

Iteration	i	$c = a + b$	**New** a	**New** b
1	2	0 + 1 = 1	1	1

2	3	1 + 1 = 2	1	2
3	4	1 + 2 = 3	2	3
4	5	2 + 3 = 5	3	5
5	6	3 + 5 = 8	5	8

The algorithm returns:

$$\boxed{8}$$

Under the fixed-size arithmetic model:

$$\boxed{\text{Time complexity} = O(n)}$$

$$\boxed{\text{Auxiliary space complexity} = O(1)}$$

Auxiliary space is the additional working memory required by an algorithm, excluding storage for the original input.

This optimized solution reduces the number of stored values without changing the number of loop iterations.

For arbitrary-precision integers, the two stored values require $O(n)$ bits near the final iteration, so the bit-space complexity is not constant.

Level 4: Algorithm Based on a Unique Insight

Some algorithmic problems require an idea that goes beyond directly applying a standard technique.

A new mathematical observation or a special property of the problem may allow a substantially more efficient solution.

Such algorithms may use:

- A newly discovered invariant.
- A special mathematical identity.
- An unusual representation of the data.
- A reduction to a different problem.
- A combination of several techniques.

An **invariant** is a property that remains true throughout specified stages of an algorithm.

For example, binary search maintains the invariant that, if the target exists, it must lie within the current search interval.

Recognizing and proving invariants is an important skill in algorithm design.

The term *magic algorithm* is sometimes used informally for a solution based on a particularly surprising insight. It is not a formal category of algorithmic complexity.

6.8 Comparing Algorithm Design Techniques

Technique	Main idea	Example	Key consideration
Naive approach	Solve the problem directly	Linear search	May perform unnecessary work
Greedy	Choose the locally best available option	Activity selection	Needs proof of optimality
Divide and conquer	Divide into smaller problems	Merge sort	Requires suitable decomposition
Dynamic programming	Reuse solutions to subproblems	Fibonacci tabulation	Benefits from overlapping subproblems
Optimization	Improve an existing algorithm	Space-optimized Fibonacci	Must preserve correctness

Chapter 6 Summary

Algorithm design involves finding a correct and efficient solution to a computational problem.

There is no universal technique that automatically produces an optimal algorithm for every problem.

Greedy algorithms, divide and conquer, and dynamic programming provide useful problem-solving strategies.

Developing algorithm design skills requires understanding these techniques, practicing their application, and learning how to justify correctness and computational complexity.

Chapter 7: Practice Problems and Solved Exercises

7.1 Exercise: Identify the Dominant Term

Find the Big-O complexity of:

$$T(n) = 8n^4 + 3n^3 + 10n^2 + 15$$

Solution

The terms are:

$$8n^4, \quad 3n^3, \quad 10n^2, \quad 15$$

The highest-degree term is:

$$8n^4$$

Ignoring the constant multiplier:

$$\boxed{T(n) = O(n^4)}$$

7.2 Exercise: Analyze a Single Loop

```
Plain text
Function Example(n):
    sum ← 0
    for i from 1 to n:
        sum ← sum + i
    return sum
```

Questions

How many times does the loop execute? What is the time complexity?

Solution

The loop starts at 1 and ends at n.

Number of iterations:

$$n - 1 + 1 = n$$

Each iteration performs one addition and one assignment under the constant-time arithmetic model.

Therefore:

$$T(n) = O(n)$$

$$\boxed{\text{Time complexity} = O(n)}$$

7.3 Exercise: Analyze Nested Loops

Plain text

```
Function Example(n):

    for i from 1 to n:

        for j from 1 to n:

            print(i, j)
```

Solution

Outer-loop iterations:

$$n$$

Inner-loop iterations per outer iteration:

$$n$$

Total executions:

$$n \times n = n^2$$

Therefore:

$$\boxed{T(n) = O(n^2)}$$

7.4 Exercise: Analyze Two Consecutive Loops

Plain text

```
Function Example(n):

    for i from 1 to n:
```

```
        print(i)

    for j from 1 to n:
        print(j)
```

Solution

First loop:

$$O(n)$$

Second loop:

$$O(n)$$

Total:

$$O(n) + O(n) = O(2n)$$

Ignoring the constant:

$$\boxed{T(n) = O(n)}$$

7.5 Exercise: Analyze a Logarithmic Loop

Plain text

```
Function Example(n):

    i ← n

    while i > 1:

        i ← floor(i / 2)
```

Dry Run for $n = 16$

Plain text

```
16 → 8 → 4 → 2 → 1
```

The loop executes four times.

Since:

$$\log_2 16 = 4$$

the time complexity is:

$$\boxed{T(n) = O(\log n)}$$

7.6 Exercise: Prove a Big-O Bound

Show that:

$$4n^3 + 7n + 9 = O(n^3)$$

Solution

For:

$$n \geq 1$$

we know:

$$n \leq n^3$$

and:

$$1 \leq n^3$$

Therefore:

$$7n \leq 7n^3$$

and:

$$9 \leq 9n^3$$

Thus:

$$4n^3 + 7n + 9 \leq 4n^3 + 7n^3 + 9n^3$$

$$= 20n^3$$

Choose:

$$c = 20, \quad N = 1$$

Therefore:

$$\boxed{4n^3 + 7n + 9 = O(n^3)}$$

7.7 Exercise: Determine a Tight Bound

Find the tight asymptotic bound for:

$$f(n) = 6n^2 + 4n + 1$$

Solution

For $n \geq 1$:

$$6n^2 + 4n + 1 \geq 6n^2$$

Also:

$$6n^2 + 4n + 1 \leq 11n^2$$

Therefore:

$$6n^2 \leq f(n) \leq 11n^2$$

Thus:

$$\boxed{f(n) = \Theta(n^2)}$$

7.8 Exercise: Determine Whether a Little-o Relationship Holds

Determine whether:

$$n \log n = o(n^2)$$

Solution

Calculate:

$$\lim_{n \to \infty} \frac{n \log n}{n^2}$$

Simplifying:

$$= \lim_{n \to \infty} \frac{\log n}{n}$$

The logarithmic function grows more slowly than the linear function.

Therefore:

$$\lim_{n \to \infty} \frac{\log n}{n} = 0$$

Hence:

$$\boxed{n \log n = o(n^2)}$$

Chapter 8: Final Review and Important Concepts

8.1 Common Complexity Classes

Complexity	Name	Example
$O(1)$	Constant	Accessing an array element
$O(\log n)$	Logarithmic	Binary search
$O(\sqrt{n})$	Square root	Testing divisors up to $\sqrt{n}$
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Two full nested loops
$O(n^3)$	Cubic	Three full nested loops

$O(2^n)$	Exponential	Generating all subsets
$O(n!)$	Factorial	Generating all permutations

The stated bounds assume standard cost models and the usual implementations of the examples.

8.2 Important Distinctions Students Must Remember

Runtime versus time complexity: Runtime may mean actual execution time; time complexity describes how computational work grows with input size.

Input value versus input length: A numerical input n may need only $O(\log n)$ bits to represent.

Loop iteration count versus operation cost: A loop executing n times is not automatically $O(n)$ if its body performs increasingly expensive operations.

Big-O versus Big-Theta: Big-O gives an upper bound, whereas Big-Theta gives a tight bound.

Big-O versus worst case: Big-O is a mathematical growth bound. Worst case describes which inputs are being analyzed.

$O(n)$ **versus** $O(n^2)$**:** Linear growth increases proportionally with input size, while quadratic growth increases proportionally to the square of the input size.

$2n$ **versus** 2^n**:** The first is linear; the second is exponential.

Time complexity versus space complexity: Time complexity measures growth of computational work; space complexity measures growth of memory requirements.

Algorithm correctness versus efficiency: A correct algorithm produces the required answer. An efficient algorithm does so using an appropriate amount of computational resources.

8.3 Self-Assessment Questions

Students should be able to answer the following questions after studying these chapters.

1. What is an algorithm, and why is algorithm analysis important?
2. What is the difference between actual runtime and theoretical time complexity?
3. Why is counting lines of code insufficient for determining exact runtime?
4. What role do the processor, compiler, and memory hierarchy play in execution time?
5. What is asymptotic analysis, and why does it ignore constant multipliers?
6. Explain the difference between linear, quadratic, and exponential growth.
7. What is the formal mathematical definition of Big-O notation?
8. Prove that $5n^2 + 3n + 7 = O(n^2)$.

9. What is the difference between Big-O, Big-Omega, Big-Theta, and little-o?
10. Explain why logarithms with different fixed bases have the same asymptotic growth.
11. Perform a complete dry run of the FibList algorithm for $n = 8$.
12. Explain why Fibonacci addition can require $O(n)$ bit operations at large indices.
13. Determine the time complexity of a program with two full nested loops.
14. Explain the difference between sequential loops and nested loops.
15. What are the main characteristics of greedy algorithms?
16. Explain the divide-and-conquer strategy with an example.
17. What are overlapping subproblems in dynamic programming?
18. Compare recursive Fibonacci with dynamic programming Fibonacci.
19. Explain the difference between a naive algorithm and an optimized algorithm.
20. Why does a better asymptotic complexity not always mean faster execution for every input size?

8.4 Concluding Discussion

Algorithm design and analysis are fundamental areas of computer science because they help us understand not only how to solve problems but also how efficiently those problems can be solved.

Exact runtime calculations are often difficult because execution time depends on hardware, software, memory systems, and the details of individual operations. Asymptotic analysis provides a mathematical way to study the growth of computational work without depending on all these details.

Big-O, Big-Omega, Big-Theta, and little-o notation provide a precise language for describing the growth of functions. These notations support the comparison of algorithms and help explain why some approaches remain practical for large inputs while others become computationally expensive.

Finally, greedy algorithms, divide and conquer, and dynamic programming provide important techniques for constructing efficient solutions. Learning these approaches, analyzing their complexity, and practicing detailed dry runs form a strong foundation for more advanced study in algorithms and data structures.